

Technical Documentation

Project Title:

ShadowLight: *Illuminating Every Step on the Web*

An AI-Powered Chrome Extension for Accessibility and Guided Web Interaction

Organization: FlairMinds Software Pvt. Ltd.

Version: 1.0

1. Technology Stack

1.1 Programming Language

- TypeScript
TypeScript is used across the project to ensure type safety and reduce runtime errors, especially when interacting with diverse and unpredictable website DOM structures.

1.2 Frontend Framework

- React.js with Vite
React.js is used to build an interactive and responsive chatbot interface, while Vite provides fast builds and an efficient development workflow.

1.3 Styling and UI Components

- Tailwind CSS and Headless UI
Tailwind CSS enables consistent and rapid styling, while Headless UI provides accessible, keyboard-friendly components aligned with accessibility standards.

1.4 Browser Extension Framework

- Plasmo
Plasmo simplifies Chrome extension development by handling Manifest V3 configuration, content scripts, and hot reloading automatically.

1.5 AI Integration

- Groq Cloud API (Llama 3)
Groq is used for natural language understanding and website summarization due to its extremely low-latency AI inference.

1.6 State Management

- TanStack Query (React Query)
Used for managing API calls, caching, loading states, and error handling in a clean and scalable manner.

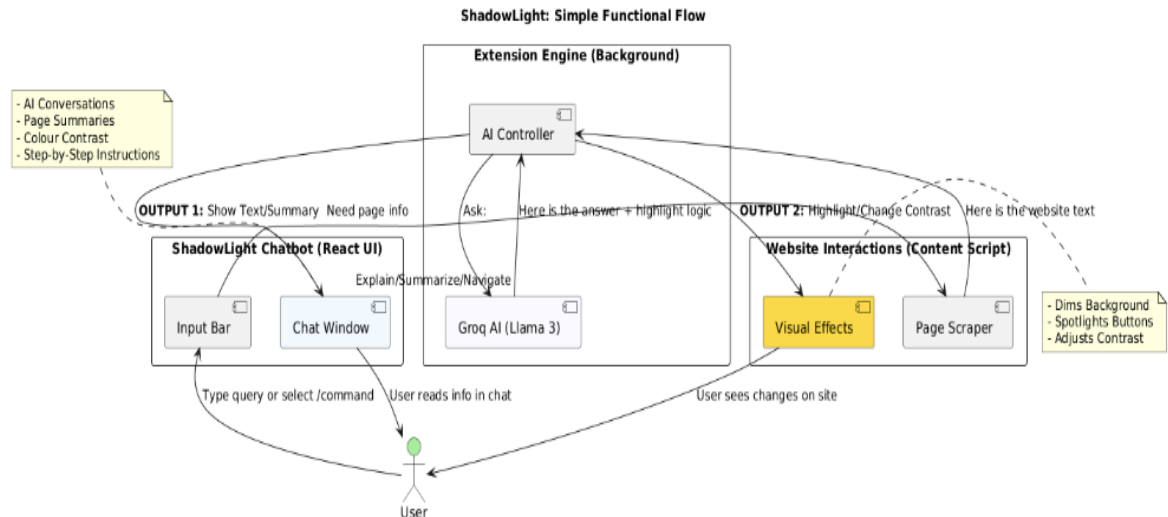
1.7 Communication

- Chrome Extension Messaging API
Enables secure and structured communication between UI components, background scripts, and content scripts.

2. Tech Stack Comparison Table

Feature	Recommended Stack (Plasmo/React/TS/Groq)	The "Vanilla" Stack (Plain JS/HTML/CSS)	The "Standard" Build (Vite/Webpack + OpenAI)
Setup Time	Low (Minutes)	Medium (Manual Manifests)	High (Configuring Build)
UI Complexity	High (React Components)	Very Hard (DOM manipulation)	High (React/Vue)
Development Speed	Fastest (Hot Reloading)	Slow (Manual Refreshing)	Fast
AI Speed	Ultra-Fast (Groq LPU)	N/A	Slow (OpenAI/Gemini)
Type Safety	Strong (TypeScript)	None	Depends
Style Isolation	Best (Shadow DOM/Tailwind)	Poor (Style Leaks)	Medium

3. Architectural Flow Diagram



3.1 User Interaction

The user enters a query or selects a command (such as *summarize the page* or *help me navigate*) through the Input Bar in the ShadowLight chatbot interface.

3.2 Context Collection

To understand the user request, the Page Scraper (content script) extracts relevant text and structural information from the active webpage and forwards it to the AI Controller in the background extension.

3.3 AI Processing

The AI Controller sends the user request and webpage content to Groq AI (Llama 3), which processes the input to generate an appropriate response along with guidance logic, such as identifying elements to highlight.

3.4 Dual Output Delivery

- Output 1 – Chat Response: The processed information is displayed in the Chat Window as summaries, explanations, or step-by-step instructions.
- Output 2 – Visual Assistance: At the same time, visual effects are applied on the webpage, including background dimming, button highlighting, and color contrast adjustment.

3.5 User Experience

The user receives guidance through the chatbot while simultaneously observing real-time visual changes on the website, resulting in a smoother, more accessible, and user-friendly browsing experience.

4. Implementation Strategy for Key Features

4.1 Website Summarization

- **Content Extraction:**
The extension extracts readable text using document, body, innerText.
- **Pre-Processing:**
non-content elements such as scripts and styles are excluded.
- **AI Prompting:**
The cleaned content is sent to Groq with a predefined system prompt to generate a short, accessible summary.

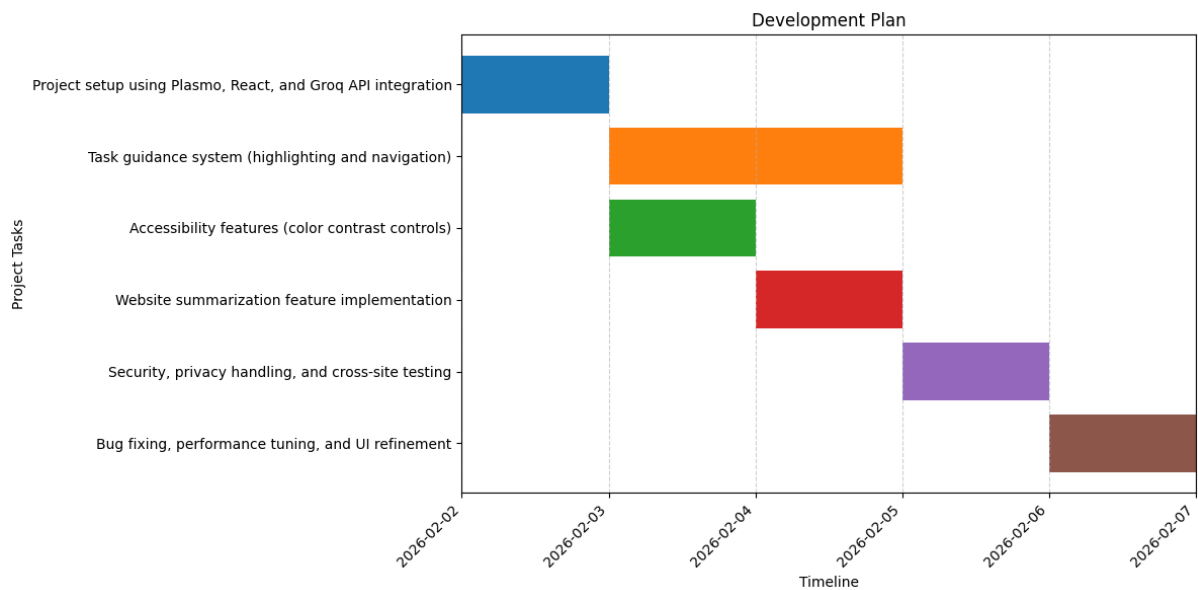
4.2 Task-Based Visual Guidance System

- **Element Detection:**
Target elements are identified using DOM selectors.
- **Screen Dimming:**
A full-screen overlay is rendered, dimming the page while keeping the target element highlighted.
- **Step Navigation:**
Users progress through steps using “Next” and “Previous” controls.
- **Acceleration Option:**
A guided tour library (e.g., driver.js) may be used to reduce development effort while maintaining flexibility.

4.3 Color Contrast Adjustment

- **Injection Method:**
A content script injects global CSS or dynamically adjusts contrast values for page elements.
- **User Control:**
Users can enable or disable the feature through the extension interface.

5. Development Timeline (1-Week Plan)



6. Final Notes

- The Llama3-8B-8192 model is recommended for summarization due to its speed and reliability.
- All AI calls are designed to be lightweight and user-initiated.
- User privacy and browser security constraints are respected throughout the implementation.